

greater than zero. If the term frequency is zero, the respective weight is also zero. Whenever the query term frequency is equal to 1, as is common on the Web, the $w_{i,q}$ weight is reduced to IDF_i . In this case we can redefine $w_{i,j}$ to be just $TF_{i,j}$, so that the sum in the numerator of formula 3.10 has terms of the form $TF \times IDF$ instead of $TF \times IDF^2$. This is the rationale behind the weighting scheme 2 in Table 3.6.

Table 3.8 illustrates the ranking computation for the collection in Figure 3.6 with regard to the query “to do.” We notice that document d_1 is the highest ranked document because it contains all query terms. Further, documents d_3 and d_4 contain just the query term “do”, but document d_4 receives a smaller rank because its vector norm is larger (effect of document length normalization). Despite its simplicity, the Vector model attains good results with general collections because of its term weighting scheme and the adoption of document length normalization.

doc	rank computation	rank
d_1	$\frac{1 \times 3 + 0.415 \times 0.830}{5.068}$	0.660
d_2	$\frac{1 \times 2 + 0.415 \times 0}{4.899}$	0.408
d_3	$\frac{1 \times 0 + 0.415 \times 1.073}{3.762}$	0.118
d_4	$\frac{1 \times 0 + 0.415 \times 1.073}{7.738}$	0.058

Table 3.8: Document ranks computed by the Vector model for the query “to do,” using TF-IDF weights given by equations 3.11 and 3.12 (see also Figures 3.2 and 3.3).

The main *advantages* of the vector model are: (1) its term-weighting scheme improves retrieval quality; (2) its partial matching strategy allows retrieval of documents that *approximate* the query conditions; (3) its cosine ranking formula sorts the documents according to their degree of similarity to the query; and (4) document length normalization is naturally built-in into the ranking. Theoretically, the vector model has the *disadvantage* that index terms are assumed to be mutually independent (equation 3.7 does *not* account for index term dependencies). However, in practice, leveraging term dependencies is challenging and might lead to poor results, if not done appropriately.

Despite its simplicity, the vector model is a resilient ranking strategy with general collections. It yields ranked answer sets which are difficult to improve upon without query expansion or relevance feedback (see Chapter 5). A large variety of alternative ranking methods have been compared to the vector model but the consensus seems to be that, with general collections, the vector model is a good and sound basic ranking method. Furthermore, it is simple and fast. For these reasons, the vector model continues to be a popular retrieval model, one which is constantly used as baseline in the evaluation of alternative ranking formulas and newly proposed IR models.

3.2.7 The Probabilistic Model

The probabilistic model, proposed in 1976 by Robertson and Sparck Jones [1365], proposes a solution to the IR problem based on a probabilistic framework. The fundamental idea is as follows. Given a user query, there is a set of documents which contains exactly the relevant documents and no other. Let us refer to this set of documents

as the *ideal* answer set. Given a description of this ideal answer set, we could retrieve the relevant documents. Thus, we can think of the querying process as a process of specifying the properties of an ideal answer set. The problem is that we do not know exactly what these properties are. All we know is that there are index terms whose semantics should be used to characterize these properties. Since these properties are not known at query time, an effort has to be made at initially guessing what they could be. This initial guess allows us to generate a preliminary probabilistic description of the ideal answer set, which can then be used to retrieve a first set of documents.

To improve the probabilistic description of the ideal answer set, an interaction with the user is initiated. For instance, the user can take a look at the retrieved documents and decide which ones are relevant and which ones are not (in truth, only the first top documents need to be examined). The system can then use this information to refine the description of the ideal answer set. By repeating this process many times, it is expected that the description of the ideal answer set becomes more accurate. Thus, one should always have in mind the need to guess at the beginning the description of the ideal answer set. Furthermore, a conscious effort is made to model this description in probabilistic terms.

Ranking for the probabilistic model is based on the following fundamental assumption stated by Robertson [1364].

Definition *Probability Ranking Principle. Given a user query q and a document d_j in the collection, the probabilistic model tries to estimate the probability that the user will find the document d_j interesting (i.e., relevant). The model assumes that this probability of relevance depends on the query and the document representations only, i.e., on the information available to the system. Further, the model assumes that there is a subset of all documents which the user prefers as the answer set for the query q . Such an ideal answer set is labeled R and should maximize the overall probability of relevance to the user. Documents in the set R are predicted to be relevant to the query. Documents not in this set are predicted to be non-relevant.*

This assumption is troublesome because relevance to the user might be affected by variables *outside* the system. Thus, the ideal answer set produced using information available to the system might not be *ideal* from the point of view of the user. Further, the principle does not state explicitly how to compute the probabilities of relevance. In fact, not even the sample space to be used for defining such probabilities is given.

Given a query q , the probabilistic model assigns to each document d_j , as a measure of its similarity to the query, the ratio $P(d_j \text{ relevant-to } q)/P(d_j \text{ non-relevant-to } q)$, which computes the odds of the document d_j being relevant to the query q . Taking the *odds of relevance* as the rank minimizes the probability of an erroneous judgement [599, 1624].

Definition *In the probabilistic model, a query q is a subset of the index terms. A document d_j is represented by a vector of binary weights indicating presence or absence of index terms as follows:*

$$\vec{d}_j = (w_{1,j}, w_{2,j}, \dots, w_{t,j})$$

where $w_{i,j} = 1$ if term k_i occurs in document d_j and $w_{i,j} = 0$, otherwise. Let R be a set of documents known (or initially guessed) to be relevant to the user, given the query q . Let $\bar{R}$ be the complement of R (i.e., the set of non-relevant documents). $P(R|\vec{d}_j, q)$

is the probability that the document d_j with representation $\vec{d}_j$ is relevant to the query q . Further, $P(\bar{R}|\vec{d}_j, q)$ is the probability that the document d_j is non-relevant to q . The similarity $\text{sim}(d_j, q)$ of the document d_j to the query q is defined as the ratio

$$\text{sim}(d_j, q) = \frac{P(R|\vec{d}_j, q)}{P(\bar{R}|\vec{d}_j, q)} \quad (3.13)$$

Using Bayes' rule,

$$\text{sim}(d_j, q) = \frac{P(\vec{d}_j|R, q) \times P(R, q)}{P(\vec{d}_j|\bar{R}, q) \times P(\bar{R}, q)} = \frac{P(\vec{d}_j|R, q) \times P(R|q)}{P(\vec{d}_j|\bar{R}, q) \times P(\bar{R}|q)}$$

$P(\vec{d}_j|R, q)$ stands for the probability that a document randomly selected from the set R of relevant documents (to query q) will have representation $\vec{d}_j$. Further, $P(R|q)$ stands for the probability that a document randomly selected from the entire collection will be found relevant to query q . The meanings attached to $P(d_j|\bar{R}, q)$ and $P(\bar{R}|q)$ are analogous and complementary.

Since $P(R|q)$ and $P(\bar{R}|q)$ are the same for all documents in the collection, we can write,

$$\text{sim}(d_j, q) \sim \frac{P(\vec{d}_j|R, q)}{P(\vec{d}_j|\bar{R}, q)} \quad (3.14)$$

Recall that the document representation $\vec{d}_j$ is composed of binary weights indicating the terms that are present and absent in document d_j . If we assume independence among the index terms, the so-called *binary independence assumption*, we can write

$$\text{sim}(d_j, q) \sim \frac{(\prod_{k_i|w_{i,j}=1} P(k_i|R, q)) \times (\prod_{k_i|w_{i,j}=0} P(\bar{k}_i|R, q))}{(\prod_{k_i|w_{i,j}=1} P(k_i|\bar{R}, q)) \times (\prod_{k_i|w_{i,j}=0} P(\bar{k}_i|\bar{R}, q))}$$

where $P(k_i|R, q)$ stands for the probability that the index term k_i is present in a document randomly selected from the set R of relevant documents (to the query q), and $P(\bar{k}_i|R, q)$ stands for the probability that the index term k_i is not present in a document randomly selected from the set R . The probabilities associated with the set $\bar{R}$ have analogous meanings.

To simplify our notation, let us adopt the following conventions

$$\begin{aligned} p_{iR} &= P(k_i|R, q) \\ q_{iR} &= P(k_i|\bar{R}, q) \end{aligned}$$

Since $P(k_i|R, q) + P(\bar{k}_i|R, q) = 1$ and $P(k_i|\bar{R}, q) + P(\bar{k}_i|\bar{R}, q) = 1$, we write

$$\text{sim}(d_j, q) \sim \frac{(\prod_{k_i|w_{i,j}=1} p_{iR}) \times (\prod_{k_i|w_{i,j}=0} (1 - p_{iR}))}{(\prod_{k_i|w_{i,j}=1} q_{iR}) \times (\prod_{k_i|w_{i,j}=0} (1 - q_{iR}))}$$

Taking logarithms, which does not change the ranking just the absolute rank values, we write

$$\begin{aligned} \text{sim}(d_j, q) \sim & \log \prod_{k_i | w_{i,j}=1} p_{iR} + \log \prod_{k_i | w_{i,j}=0} (1 - p_{iR}) \\ & - \log \prod_{k_i | w_{i,j}=1} q_{iR} - \log \prod_{k_i | w_{i,j}=0} (1 - q_{iR}) \end{aligned}$$

Summing up terms that cancel each other, we obtain

$$\begin{aligned} \text{sim}(d_j, q) \sim & \log \prod_{k_i | w_{i,j}=1} p_{iR} + \log \prod_{k_i | w_{i,j}=0} (1 - p_{iR}) \\ & - \log \prod_{k_i | w_{i,j}=1} (1 - p_{iR}) + \log \prod_{k_i | w_{i,j}=1} (1 - p_{iR}) \\ & - \log \prod_{k_i | w_{i,j}=1} q_{iR} - \log \prod_{k_i | w_{i,j}=0} (1 - q_{iR}) \\ & + \log \prod_{k_i | w_{i,j}=1} (1 - q_{iR}) - \log \prod_{k_i | w_{i,j}=1} (1 - q_{iR}) \end{aligned}$$

Which can be rearranged using logarithm operations as

$$\begin{aligned} \text{sim}(d_j, q) \sim & \log \prod_{k_i | w_{i,j}=1} \frac{p_{iR}}{(1 - p_{iR})} + \log \prod_{k_i} (1 - p_{iR}) \\ & + \log \prod_{k_i | w_{i,j}=1} \frac{(1 - q_{iR})}{q_{iR}} - \log \prod_{k_i} (1 - q_{iR}) \end{aligned}$$

Notice that there are two terms that are products over *all* index terms, which means that they are independent of the query q and of the document d_j . That is, they are constants and can be disregarded for the purpose of ranking. Further, assuming that

$$\forall k_i \notin q, \quad p_{iR} = q_{iR}$$

and converting the log products into sums of logs, we finally obtain

$$\text{sim}(d_j, q) \sim \sum_{k_i \in q \wedge k_i \in d_j} \log \left(\frac{p_{iR}}{1 - p_{iR}} \right) + \log \left(\frac{1 - q_{iR}}{q_{iR}} \right) \quad (3.15)$$

which is a key expression for ranking computation in the probabilistic model.

Since we do not know the set R at the beginning, it is necessary to devise a method for initially computing the probabilities $p_{i,R}$ and $q_{i,R}$. There are many alternatives for such computation, as we now discuss.

Term Incidence Contingency Table

A more detailed interpretation of the probabilistic ranking formula can be done using a contingency table [1365] as follows.

Definition Let N be the number of documents in the collection and n_i be the number of documents that contain term k_i , as before. Further, let R be the total number of relevant documents to query q (in the opinion of the users) and r_i be the number of relevant documents that contain term k_i . The term incidence contingency table is given in Table 3.9.

Case	relevant	non-relevant	total
Docs that contain k_i	r_i	$n_i - r_i$	n_i
Docs that do not contain k_i	$R - r_i$	$N - n_i - (R - r_i)$	$N - n_i$
All docs	R	$N - R$	N

Table 3.9: Contingency table.

If the information in the term incidence contingency table were available for any given query (it is not, given that we do not know the relevant docs for a new query), we could write

$$p_{iR} = \frac{r_i}{R}, \quad q_{iR} = \frac{n_i - r_i}{N - R}$$

Then, equation 3.15 could be rewritten as

$$\text{sim}(d_j, q) \sim \sum_{k_i[q, d_j]} \log \left(\frac{r_i(N - n_i - R + r_i)}{(R - r_i)(n_i - r_i)} \right) \quad (3.16)$$

where $k_i[q, d_j]$ is a short notation for $k_i \in q \wedge k_i \in d_j$. For this formula to work, we are still dependent on estimating what are the relevant documents for the query. For handling small values of r_i , it is convenient to add 0.5 to each of the terms in the formula above, which yields

$$\text{sim}(d_j, q) \sim \sum_{k_i[q, d_j]} \log \left(\frac{(r_i + 0.5)(N - n_i - R + r_i + 0.5)}{(R - r_i + 0.5)(n_i - r_i + 0.5)} \right)$$

This formula is well-behaved in extreme conditions and does not lead to infinite rank values under particular estimates of r_i and R , such as $R = r_i$. Considered as the classic ranking equation for the probabilistic model, it is also known as the Robertson-Sparck Jones equation.

The equation above cannot be computed without estimates of r_i and R . One possibility is to assume $R = r_i = 0$, as a way to bootstrap the ranking equation, which leads to

$$\text{sim}(d_j, q) \sim \sum_{k_i \in q \wedge k_i \in d_j} \log \left(\frac{N - n_i + 0.5}{n_i + 0.5} \right) \quad (3.17)$$

In the absence of relevance information, this is the equation for ranking computation in the probabilistic model. Notice that, while an IDF component is present, there is no provision for neither TF weights nor document length normalization. These problems will be corrected in the BM25 model, as we later discuss.

doc	rank computation	rank
d_1	$\log \frac{4-2+0.5}{2+0.5} + \log \frac{4-3+0.5}{3+0.5}$	- 1.222
d_2	$\log \frac{4-2+0.5}{2+0.5}$	0
d_3	$\log \frac{4-3+0.5}{3+0.5}$	- 1.222
d_4	$\log \frac{4-3+0.5}{3+0.5}$	- 1.222

Table 3.10: Document ranks computed by equation 3.17 for the query “to do” and the example collection in Figure 3.6.

Table 3.10 illustrates the ranking computation for the query “to do” and the example collection in Figure 3.6. The probabilistic ranking equation does not behave well in this case because the weight produced by the term “do” is negative. That is, equation 3.17 does not behave well whenever $n_i > N/2$, in which case negative terms are introduced in the computation.

One alternative for avoiding this anomalous behavior is to drop the n_i factor from the numerator of equation 3.17, as suggested in [1374, 1371]. This leads to

$$\text{sim}(d_j, q) \sim \sum_{k_i \in q \wedge k_i \in d_j} \log \left(\frac{N + 0.5}{n_i + 0.5} \right) \quad (3.18)$$

Notice that now a term that occurs in all documents ($n_i = N$) produces a weight equal to zero and that there are no negative term weights. Using this formulation, we redo the ranking computation for our example collection as illustrated in Table 3.11. The probabilistic ranking now resembles the ordering generated by the vector model i.e., $d_1 > d_2 > d_3 = d_4$. It cannot distinguish between documents d_3 and d_4 because, contrary to the vector model, it includes no provision for document length normalization.

doc	rank computation	rank
d_1	$\log \frac{4+0.5}{2+0.5} + \log \frac{4+0.5}{3+0.5}$	1.210
d_2	$\log \frac{4+0.5}{2+0.5}$	0.847
d_3	$\log \frac{4+0.5}{3+0.5}$	0.362
d_4	$\log \frac{4+0.5}{3+0.5}$	0.362

Table 3.11: Document ranks computed by the modified probabilistic equation 3.18 for the query “to do”.

Our examples above considered that $r_i = R = 0$. One alternative to estimate the parameters r_i and R more carefully is to perform an initial search using the equation above, select the top 10 – 20 ranked documents, inspect them to gather new estimates for r_i and R , remove these 10 – 20 documents from the collection, and then rerun the query with the estimates obtained for r_i and R – a procedure called *residual ranking* [708]. Unfortunately, procedures such as these are not practical since they require human intervention to initially select the relevant documents.

Ranking in the Absence of Relevance Information

Equation 3.17 is the basic equation for ranking computation in the probabilistic model, when no relevance information is provided. If relevance information is to be considered, then human intervention is required. Croft and Harper [452] have proposed to avoid human intervention by devising automatic procedures for refining the estimates of parameters r_i and R , as we now discuss.

To use equation 3.15 in the absence of relevance information we can (a) assume that p_{iR} is constant for all index terms k_i , typically equal to 0.5, indicating no prior knowledge of its distribution, and (b) assume that the distribution of index terms among the non-relevant documents can be approximated by the distribution of index terms among all the documents in the collection. These two assumptions yield

$$p_{iR} = 0.5; \quad q_{iR} = \frac{n_i}{N}$$

Substituting into equation 3.15, we obtain

$$\text{sim}(d_j, q) \sim \sum_{k_i \in q \wedge k_i \in d_j} \log \left(\frac{N - n_i}{n_i} \right) \quad (3.19)$$

which is the same as equation 3.17, without the 0.5 correction factor. Given this initial guess, we can retrieve documents that contain the query terms and also provide an initial probabilistic ranking for them. After that, we can attempt to improve this initial ranking as follows.

Let D be a subset of the documents initially retrieved and ranked by the probabilistic model. Such a subset can be defined, for instance, as the top M ranked documents, in which M is a previously defined threshold. Further, let D_i be the subset of D composed of the documents in D which contain the index term k_i . For simplicity, we also use D and D_i to refer to the number of elements in these sets. To improve the ranking, we need to improve our guesses for p_{iR} and q_{iR} . This can be attempted with the following assumptions: (a) we can approximate p_{iR} by the distribution of the index term k_i among the documents retrieved so far, and (b) we can approximate q_{iR} by considering that all the non-retrieved documents are not relevant. With these assumptions, we can write,

$$p_{iR} = \frac{D_i}{D}; \quad q_{iR} = \frac{n_i - D_i}{N - D}$$

This process can then be repeated recursively. By doing so, we expect to improve on our guesses for the probabilities p_{iR} and q_{iR} without any assistance from a human subject (contrary to the original idea). Of course, this is totally dependent on the patterns of index term occurrence, which are noisy (index terms by themselves carry little semantics and might be misleading). Thus, the approach does not necessarily lead to improved results as the number of iterations increases. One alternative is to use assistance from the user in the definition of the subset D (as originally conceived).

The last formulas for p_{iR} and q_{iR} pose problems for small values of D and D_i that arise in practice such as $D = 1$ and $D_i = 0$. To circumvent these problems, adjustment factors can be added in, which yields

$$p_{iR} = \frac{D_i + 0.5}{D + 1}; \quad q_{iR} = \frac{n_i - D_i + 0.5}{N - D + 1}$$

An adjustment factor which is constant and equal to 0.5 might be limiting. An alternative is to take the fraction n_i/N as the adjustment factor which yields

$$p_{iR} = \frac{D_i + \frac{n_i}{N}}{D + 1}; \quad q_{iR} = \frac{n_i - D_i + \frac{n_i}{N}}{N - D + 1}$$

In this case, the ranking computation is done in full automatic fashion, with no relevance information provided.

The ranking formulas above do not work well with our example collection of Figure 3.6 because the values of N are too close to the values of n_i (for instance, some ranks become negative). With larger and more realistic collections, the formulas work fine.

Advantages and Disadvantages of the Probabilistic Model

The main *advantage* of the probabilistic model, in theory, is its optimality, i.e., documents are ranked in decreasing order of their *probability* of being relevant, based on the information available to the system. In practice, however, this does not work so well because relevance of a document is affected by variables that are not in the system. The *disadvantages* include: (1) the need to guess the initial separation of documents into relevant and non-relevant sets; (2) the fact that the method does *not* take into account the frequency with which an index term occurs inside a document (i.e., all weights are binary); and (3) the lack of document length normalization. More advanced variations of the probabilistic model, such as the BM-25 model, correct these deficiencies to yield improved retrieval.

3.2.8 Brief Comparison of Classic Models

The Boolean model is considered to be the weakest classic method. Its main problem is the lack of partial matching between the query and the documents, which frequently leads to poor quality. There is some controversy as to whether the probabilistic model outperforms the vector model. Croft performed some experiments and suggested that the probabilistic model provides a better retrieval quality [452]. However, experiments done afterwards by Salton and Buckley refute that claim [1410]. Through several different measures, Salton and Buckley showed that the vector model is *expected* to outperform the probabilistic model with general collections. This also seems to be the dominant thought among researchers and practitioners of IR.

With general collections, the vector model provides a reasonable and robust IR model to compare with. To better appreciate this point, one can run the following exercise. Take a document collection with dozens of thousands of documents and use it as reference. Devise a new ranking formula, distinct from the ones we have discussed so far. Following, implement the standard vector model. Test and compare the ranking provided by the vector model with the ranking provided by the new ranking formula. This can be done by running a preselected set of queries on the collection and comparing the answers generated by the two ranking formulas (we discuss how to run comparative evaluations in Chapter 4). Try to improve on the new ranking formula to make its results comparable to those of the vector model. Quite interestingly, this is not simple or easy to do. The vector model, whose weighting scheme is firmly grounded on information theory, provides a simple and yet effective ranking formula for general collections.